Sesión 12: BASES DE DATOS VECTORIALES Y RAG

Fundamentos, tecnologías y aplicaciones en IA

Por

Carlos César Sánchez Coronel

2026

Índice general

Índice general **2**

1. Introducción a las Bases de Datos Vectoriales **3**
1.1. ¿Qué es un vector en el contexto de IA? . . . 3
1.2. Limitaciones de las bases de datos tradicionales . . . 3
1.3. ¿Qué hace una base de datos vectorial? . . . 3

2. Embeddings: El Corazón de la Búsqueda Semántica **5**
2.1. Definición y generación . . . 5
2.1.1. Ejemplo con Sentence Transformers en Python . . . 5
2.2. Visualización de embeddings . . . 5

3. Búsqueda de Similitud: k-NN y ANN **6**
3.1. k-NN exacto . . . 6
3.2. Aproximación: ANN (Approximate Nearest Neighbors) . . . 6
3.2.1. HNSW (Hierarchical Navigable Small World) . . . 6
3.2.2. IVF (Inverted File Index) . . . 6
3.2.3. PQ (Product Quantization) . . . 6
3.2.4. Comparativa . . . 7

4. Métricas de Distancia **8**
4.1. Similitud de Coseno . . . 8
4.2. Distancia Euclidiana (L2) . . . 8
4.3. Producto Punto . . . 8
4.4. Qué métrica elegir . . . 8

5. Tecnologías de Bases de Datos Vectoriales **9**
5.1. Bases de Datos Nativas . . . 9
5.1.1. Pinecone . . . 9
5.1.2. Milvus . . . 9
5.1.3. Weaviate . . . 9
5.1.4. Qdrant . . . 9
5.1.5. Chroma . . . 9
5.2. Extensiones sobre Bases de Datos Existentes . . . 10
5.2.1. pgvector . . . 10
5.2.2. Elasticsearch . . . 10
5.2.3. Redis Stack . . . 10
5.2.4. MongoDB . . . 10
5.3. Tabla comparativa . . . 10

6. Retrieval Augmented Generation (RAG) 11
6.1. Problema que resuelve . . . 11
6.2. Flujo de trabajo típico . . . 11
6.3. Orquestadores . . . 12
6.3.1. LangChain . . . 12
6.3.2. LlamaIndex . . . 12
6.3.3. Haystack . . . 13

7. Integración con SQL y NoSQL 14
7.1. Casos de integración . . . 14
7.2. Estrategias de consistencia . . . 14
7.3. Ejemplo con pgvector y PostgreSQL . . . 14

8. Despliegue On-Premise y Cloud 16
8.1. On-Premise . . . 16
8.2. Cloud . . . 16
8.3. Dimensionamiento y costos . . . 16

9. Casos de Uso Reales 18
9.1. Búsqueda semántica en comercio electrónico . . . 18
9.2. Sistemas de preguntas y respuestas (FAQ) . . . 18
9.3. Recomendación de contenidos . . . 18
9.4. Detección de plagio o documentos duplicados . . . 18
9.5. Búsqueda multimodal . . . 18

10. Ejercicios Resueltos 19
10.1. Ejercicio 1: Generar embeddings y buscar similitud con pgvector . . . 19
10.2. Ejercicio 2: Implementar un pipeline RAG con LangChain y Chroma . . . 20
10.3. Ejercicio 3: Búsqueda híbrida con filtros en pgvector . . . 21
10.4. Ejercicio 4: Comparación de índices en Milvus . . . 21

11. Glosario de Términos 23

Referencias 24

Capítulo 1

Introducción a las Bases de Datos Vectoriales

Las bases de datos tradicionales (SQL y NoSQL) están optimizadas para búsquedas por coincidencia exacta de valores o rangos. Sin embargo, en el contexto de la inteligencia artificial, necesitamos buscar por **similitud semántica**: encontrar elementos que sean conceptualmente similares, aunque no compartan palabras clave exactas. Por ejemplo, buscar "mascota felinaz querer obtener resultados sobre "gatos". Aquí es donde entran las **bases de datos vectoriales**.

1.1. ¿Qué es un vector en el contexto de IA?

Un vector es una lista ordenada de números (coordenadas) que representa un objeto (texto, imagen, audio, etc.) en un espacio de alta dimensionalidad. Estos vectores, llamados **embeddings**, son generados por modelos de machine learning (como Word2Vec, BERT, CLIP) de manera que objetos semánticamente cercanos tienen vectores cercanos según alguna métrica de distancia.

1.2. Limitaciones de las bases de datos tradicionales

- No pueden buscar por similitud semántica de forma eficiente.
- Las consultas de texto (LIKE, búsqueda de texto completo) solo encuentran coincidencias exactas o por palabras clave, no capturan el significado.
- No están optimizadas para manejar vectores de cientos o miles de dimensiones.

1.3. ¿Qué hace una base de datos vectorial?

Una base de datos vectorial está diseñada para:

- Almacenar vectores junto con metadatos asociados.
- Indexar los vectores para permitir búsquedas rápidas de los vecinos más cercanos (Approximate Nearest Neighbors, ANN).

- Ofrecer APIs para calcular distancias y recuperar los elementos más similares a un vector de consulta.

Capítulo 2

Embeddings: El Corazón de la Búsqueda Semántica

2.1. Definición y generación

Un embedding es una representación densa de baja dimensionalidad (comparada con el espacio original) de un objeto. Por ejemplo, un texto de longitud variable se convierte en un vector de, digamos, 384, 768 o 1536 dimensiones.

Modelos populares para generar embeddings:

- **Texto**: Sentence Transformers (all-MiniLM-L6-v2), OpenAI embeddings (text-embedding-ada-002), BERT, RoBERTa.
- **Imágenes**: CLIP, ResNet, VGG.
- **Múltiples modalidades**: CLIP (imagen y texto), ImageBind.

2.1.1. Ejemplo con Sentence Transformers en Python

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
oraciones = ["Un gato en la alfombra", "Un perro jugando en el
   parque"]
embeddings = model.encode(oraciones)
print(embeddings.shape)  # (2, 384)
print(embeddings[0][:5]) # Primeros 5 valores del primer vector
```

2.2. Visualización de embeddings

Aunque no podemos visualizar espacios de alta dimensionalidad, podemos reducirlos a 2D con técnicas como PCA o t-SNE para inspección. Esto ayuda a ver cómo se agrupan elementos semánticamente similares.

Capítulo 3

Búsqueda de Similitud: k-NN y ANN

3.1. k-NN exacto

El vecino más cercano exacto (k-NN) consiste en calcular la distancia entre el vector de consulta y todos los vectores almacenados, y seleccionar los k más cercanos. Esto tiene complejidad O(n) por consulta, inviable para millones de vectores.

3.2. Aproximación: ANN (Approximate Nearest Neighbors)

Los algoritmos ANN sacrifican un poco de precisión para lograr búsquedas en tiempo sublineal (logarítmico o incluso constante). Los más comunes:

3.2.1. HNSW (Hierarchical Navigable Small World)

Crea una estructura de grafos multinivel para navegar rápidamente hacia los vecinos. Es uno de los algoritmos más populares por su equilibrio entre velocidad y precisión. Usado en Milvus, Pinecone, Weaviate, pgvector (con extensión).

3.2.2. IVF (Inverted File Index)

Divide el espacio en clusters (mediante k-means) y asigna cada vector a su cluster. En la búsqueda, solo se exploran los clusters más cercanos a la consulta. Usado en Faiss (Facebook AI Similarity Search).

3.2.3. PQ (Product Quantization)

Comprime vectores dividiéndolos en subvectores y cuantificando cada subespacio. Reduce drásticamente el uso de memoria y acelera la búsqueda.

3.2.4. Comparativa

Algoritmo	**Velocidad**	**Precisión**
HNSW	Alta	Muy alta
IVF	Alta	Alta (depende de número de clusters)
PQ	Muy alta	Media

Capítulo 4

Métricas de Distancia

La noción de çercanía”depende de la métrica elegida. Las más comunes en espacios de embeddings:

4.1. Similitud de Coseno

Mide el coseno del ángulo entre dos vectores. Ignora la magnitud, se enfoca en la dirección. Es la más usada para texto.

$$\text{similitud} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$$

Valores entre -1 y 1 (o 0 y 1 si los vectores son no negativos). Cuanto más cercano a 1, más similares.

4.2. Distancia Euclidiana (L2)

Distancia geométrica tradicional. Sensible a la magnitud.

$$d = \sqrt{\sum_i (u_i - v_i)^2}$$

4.3. Producto Punto

Si los vectores están normalizados (norma 1), el producto punto equivale a la similitud de coseno. Algunas bases de datos lo usan por eficiencia.

4.4. Qué métrica elegir

Depende del modelo de embeddings. Por ejemplo, los modelos de Sentence Transformers suelen entrenarse con similitud de coseno. Es importante usar la métrica para la que fue entrenado el modelo.

Capítulo 5

Tecnologías de Bases de Datos Vectoriales

Existen dos categorías: bases de datos nativas (diseñadas exclusivamente para vectores) y extensiones sobre bases de datos existentes.

5.1. Bases de Datos Nativas

5.1.1. Pinecone

Servicio en la nube totalmente gestionado. Ofrece indexación automática, escalabilidad y búsqueda de alta precisión. Muy fácil de usar. No tiene versión on-premise.

5.1.2. Milvus

Plataforma de código abierto para búsqueda vectorial. Escalable, soporta múltiples índices (HNSW, IVF, etc.), y se puede desplegar on-premise o en la nube. Tiene versiones gestionadas (Zilliz Cloud).

5.1.3. Weaviate

Base de datos vectorial de código abierto con soporte para módulos de ML (transformers, etc.). Permite búsqueda híbrida (vectorial + filtros escalares).

5.1.4. Qdrant

Escrita en Rust, enfocada en alto rendimiento. Ofrece filtrado y búsqueda vectorial. Disponible como código abierto y servicio gestionado.

5.1.5. Chroma

Ligera, pensada para prototipos y proyectos pequeños. Muy fácil de usar desde Python.

5.2. Extensiones sobre Bases de Datos Existentes

5.2.1. pgvector

Extensión para PostgreSQL que añade un tipo de dato vector y soporte para índices (IVFFlat, HNSW). Permite integrar búsqueda vectorial con datos relacionales.

```
CREATE EXTENSION vector;
CREATE TABLE items (id bigserial, embedding vector(384));
CREATE INDEX ON items USING ivfflat (embedding vector_cosine_ops)
   ;
SELECT * FROM items ORDER BY embedding <=> '[0.1,0.2,...]' LIMIT
   5;
```

5.2.2. Elasticsearch

Desde la versión 7.x, soporta búsqueda vectorial (dense vector) con similitud de coseno o producto punto. Permite combinar con búsqueda de texto completo.

5.2.3. Redis Stack

Redis añadió RedisVL (módulo de búsqueda vectorial) y soporte para índices de vectores. Útil para caché y aplicaciones en tiempo real.

5.2.4. MongoDB

A través de agregaciones, se puede hacer búsqueda vectorial simple, pero no está optimizado para grandes escalas. Existe una extensión de Atlas Search que soporta vectores.

5.3. Tabla comparativa

Tecnología	Tipo	Índices	Despliegue
Pinecone	Nativa (gestionada)	HNSW	Solo cloud
Milvus	Nativa (open source)	HNSW, IVF, etc.	On-premise / cloud
Weaviate	Nativa (open source)	HNSW	On-premise / cloud
pgvector	Extensión PostgreSQL	IVFFlat, HNSW	On-premise / cloud
Elasticsearch	Motor de búsqueda	HNSW (plugin)	On-premise / cloud
RedisVL	Módulo Redis	HNSW	On-premise / cloud

Capítulo 6

Retrieval Augmented Generation (RAG)

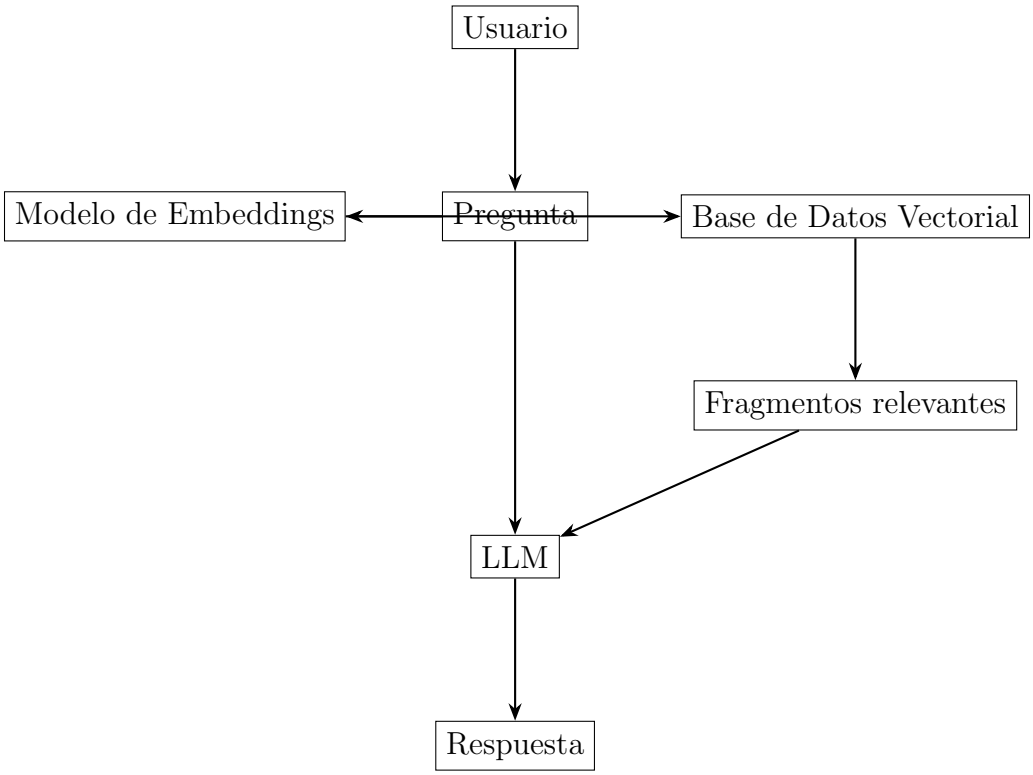
RAG es un patrón arquitectónico que combina un sistema de recuperación (retriever) con un modelo generativo (LLM) para producir respuestas basadas en información externa y actualizada.

6.1. Problema que resuelve

Los LLMs tienen un conocimiento limitado a su fecha de entrenamiento y pueden alucinar (inventar) cuando no saben algo. RAG les proporciona contexto relevante extraído de una base de conocimiento, reduciendo alucinaciones y permitiendo respuestas sobre datos privados o recientes.

6.2. Flujo de trabajo típico

1. **Indexación**: Se dividen los documentos en fragmentos (chunks), se generan sus embeddings y se almacenan en una base de datos vectorial junto con el texto original y metadatos.
2. **Consulta**: El usuario hace una pregunta.
3. **Embedding de consulta**: Se genera el embedding de la pregunta usando el mismo modelo que se usó para indexar.
4. **Búsqueda de similitud**: La base de datos vectorial devuelve los k fragmentos más similares.
5. **Generación de contexto**: Se construye un prompt que incluye los fragmentos recuperados y la pregunta original.
6. **Llamada al LLM**: El modelo genera una respuesta basada en el contexto.



6.3. Orquestadores

Para implementar RAG, se utilizan frameworks que simplifican la integración:

6.3.1. LangChain

Proporciona herramientas para cargar documentos, dividirlos, generar embeddings, interactuar con bases de datos vectoriales y LLMs.

```
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Pinecone
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Pinecone.from_documents(docs, embeddings,
    index_name="mi-index")
qa = RetrievalQA.from_chain_type(llm=OpenAI(), retriever=
    vectorstore.as_retriever())
respuesta = qa.run("  Cul   es la capital de Francia?")
```

6.3.2. LlamaIndex

Similar a LangChain, pero con un enfoque más centrado en índices y estructuras de datos.

6.3.3. Haystack

Framework de código abierto para construir sistemas de búsqueda y RAG.

Capítulo 7

Integración con SQL y NoSQL

En aplicaciones reales, las bases de datos vectoriales no viven aisladas. A menudo coexisten con bases de datos relacionales y NoSQL.

7.1. Casos de integración

- Almacenar metadatos (precio, categoría, fecha) en PostgreSQL y embeddings en pgvector, permitiendo búsquedas híbridas.
- Usar MongoDB para documentos y una base vectorial para búsqueda semántica sobre esos documentos.
- Tener un data lake con archivos Parquet y usar un catálogo para indexar embeddings.

7.2. Estrategias de consistencia

- **Actualización síncrona**: Al insertar un documento, se genera su embedding y se guarda en ambas bases.
- **CDC (Change Data Capture)**: Detectar cambios en la BD principal y actualizar la vectorial mediante eventos (Kafka, Debezium).

7.3. Ejemplo con pgvector y PostgreSQL

```sql
-- Tabla de productos con embedding
CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    nombre TEXT,
    descripcion TEXT,
    precio NUMERIC,
    embedding vector(384)
);

--  ndice  HNSW (requiere pgvector >= 0.5.0)
```

```sql
CREATE INDEX ON productos USING hnsw (embedding vector_cosine_ops
   );

-- B squeda: productos similares a "zapatos deportivos" con
   precio < 100
SELECT nombre, precio
FROM productos
WHERE precio < 100
ORDER BY embedding <=> (SELECT embedding FROM productos WHERE id
   = 123)
LIMIT 10;
```

Capítulo 8

Despliegue On-Premise y Cloud

8.1. On-Premise

Empresas con requisitos de seguridad o datos sensibles pueden desplegar bases de datos vectoriales en sus propios servidores.

- Milvus, Weaviate, Qdrant tienen versiones open source que se pueden instalar en Kubernetes o máquinas virtuales.
- pgvector se instala como extensión de PostgreSQL en cualquier servidor.

Ventajas: control total, sin costos de salida de datos. Desventajas: mantenimiento, escalado manual.

8.2. Cloud

Los proveedores ofrecen servicios gestionados:

- **Pinecone**: totalmente gestionado, sin preocuparse por infraestructura.
- **Milvus Cloud** (Zilliz), **Weaviate Cloud**, **Qdrant Cloud**.
- **AWS**: OpenSearch Serverless con vectorial, MemoryDB para Redis (pronto), y soporte en Aurora PostgreSQL (pgvector).
- **Google Cloud**: Vertex AI Vector Search (antes Matching Engine).
- **Azure**: Cognitive Search con vectorial, Cosmos DB con extensión de vectores (en preview).

Ventajas: escalabilidad automática, menor overhead operativo. Desventajas: costos variables, dependencia del proveedor.

8.3. Dimensionamiento y costos

El costo depende de:

- Número de vectores y su dimensionalidad.

- Tipo de índice (HNSW consume más memoria pero es más rápido; IVF consume menos pero puede ser más lento).
- Operaciones de escritura y consultas por segundo.

Ejemplo: Pinecone cobra por hora y por la cantidad de vectores almacenados. Para 1 millón de vectores de 384 dimensiones, el costo aproximado es de unos $0.50 por hora en el plan estándar. En cambio, pgvector en una instancia de AWS RDS puede ser más económico si ya se tiene la base de datos.

Capítulo 9

Casos de Uso Reales

9.1. Búsqueda semántica en comercio electrónico

Un sitio de ventas puede permitir a los usuarios buscar productos por descripción semántica: "vestido rojo elegante.en lugar de palabras clave exactas. Los embeddings de las descripciones se comparan con el embedding de la consulta.

9.2. Sistemas de preguntas y respuestas (FAQ)

Una empresa puede tener cientos de documentos internos. Un chatbot RAG responde preguntas de empleados buscando en esos documentos.

9.3. Recomendación de contenidos

En una plataforma de streaming, se pueden generar embeddings de películas (basados en trama, género, actores) y recomendar las más similares a las que ha visto el usuario.

9.4. Detección de plagio o documentos duplicados

Comparando embeddings de documentos, se pueden encontrar similitudes entre textos.

9.5. Búsqueda multimodal

Con modelos como CLIP, se pueden buscar imágenes usando texto o viceversa. Las imágenes y textos se proyectan al mismo espacio vectorial.

Capítulo 10

Ejercicios Resueltos

10.1. Ejercicio 1: Generar embeddings y buscar similitud con pgvector

Enunciado: Usando Python y Sentence Transformers, genera embeddings para una lista de oraciones y almacénalos en PostgreSQL con pgvector. Luego realiza una consulta de búsqueda semántica.

Solución:

```
import psycopg2
from sentence_transformers import SentenceTransformer

# Conectar a PostgreSQL
conn = psycopg2.connect(dbname="test", user="postgres")
cur = conn.cursor()
cur.execute("CREATE EXTENSION IF NOT EXISTS vector;")
cur.execute("DROP TABLE IF EXISTS oraciones;")
cur.execute("CREATE TABLE oraciones (id SERIAL PRIMARY KEY, texto
    TEXT, embedding vector(384));")

# Modelo de embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')
oraciones = [
    "El gato duerme en el sof ",
    "Los perros juegan en el parque",
    "Me gusta la comida italiana",
    "El cielo est  nublado hoy"
]
embeddings = model.encode(oraciones).tolist()

# Insertar
for texto, emb in zip(oraciones, embeddings):
    cur.execute("INSERT INTO oraciones (texto, embedding) VALUES
   (%s, %s);", (texto, emb))
conn.commit()

# Consulta: buscar oraciones similares a "animales dom sticos"
consulta = "animales dom sticos"
```

```
emb_consulta = model.encode([consulta]).tolist()[0]
cur.execute("""
    SELECT texto, embedding <=> %s::vector AS distancia
    FROM oraciones
    ORDER BY embedding <=> %s::vector
    LIMIT 2;
""", (emb_consulta, emb_consulta))
resultados = cur.fetchall()
for texto, dist in resultados:
    print(f"{texto} (distancia: {dist:.4f})")
cur.close()
conn.close()
```

Salida esperada: las oraciones sobre gato y perro serán las más cercanas.

10.2. Ejercicio 2: Implementar un pipeline RAG con LangChain y Chroma

Enunciado: Cargar un documento PDF, dividirlo en fragmentos, crear embeddings y construir un sistema de preguntas y respuestas usando LangChain y Chroma como base vectorial.

Solución:

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import
   RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA

# 1. Cargar y dividir documento
loader = PyPDFLoader("documento.pdf")
documentos = loader.load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500,
   chunk_overlap=50)
docs = text_splitter.split_documents(documentos)

# 2. Crear embeddings y vectorstore
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Chroma.from_documents(docs, embeddings)

# 3. Configurar LLM (OpenAI) - requiere API key
llm = OpenAI(temperature=0)

# 4. Crear cadena de QA
qa = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.
   as_retriever())

# 5. Preguntar
respuesta = qa.run("  Cul   es el tema principal del documento?")
```

```
print(respuesta)
```

10.3. Ejercicio 3: Búsqueda híbrida con filtros en pgvector

Enunciado: En una tabla de productos con precio y categoría, buscar productos similares a una descripción dada, filtrando por precio mayor a 50 y categoría .electrónica".

Solución:

```
import psycopg2
from sentence_transformers import SentenceTransformer

conn = psycopg2.connect(dbname="test", user="postgres")
cur = conn.cursor()
model = SentenceTransformer('all-MiniLM-L6-v2')

# Suponiendo que ya existen productos con embeddings
consulta = "tel fono inteligente con buena c mara"
emb_consulta = model.encode([consulta]).tolist()[0]

cur.execute("""
    SELECT nombre, precio, embedding <=> %s::vector AS dist
    FROM productos
    WHERE precio > 50 AND categoria = 'electronica'
    ORDER BY embedding <=> %s::vector
    LIMIT 5;
""", (emb_consulta, emb_consulta))
for row in cur.fetchall():
    print(row)
cur.close()
conn.close()
```

10.4. Ejercicio 4: Comparación de índices en Milvus

Enunciado: Usando Milvus (modo local), crear una colección, insertar vectores aleatorios y comparar el rendimiento de búsqueda con índice HNSW vs IVFFlat.

Solución:

```
from pymilvus import connections, Collection, FieldSchema,
   CollectionSchema, DataType, utility
import numpy as np
import time

connections.connect("default", host="localhost", port="19530")

# Crear colecci n
fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True)
   ,
```

```
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR,
   dim=128)
]
schema = CollectionSchema(fields)
collection = Collection("test_vector", schema)

# Insertar 10000 vectores aleatorios
data = [
    [i for i in range(10000)],
    [np.random.rand(128).tolist() for _ in range(10000)]
]
collection.insert(data)
collection.flush()

#  ndice  HNSW
hnsw_params = {"index_type": "HNSW", "metric_type": "COSINE", "
   params": {"M": 16, "efConstruction": 200}}
collection.create_index("embedding", hnsw_params)
collection.load()
start = time.time()
results = collection.search([np.random.rand(128).tolist()], "
   embedding", param={"metric_type": "COSINE", "params": {"ef":
   64}}, limit=10)
print("HNSW tiempo:", time.time() - start)

#  ndice  IVF
ivf_params = {"index_type": "IVF_FLAT", "metric_type": "COSINE",
   "params": {"nlist": 128}}
collection.drop_index()
collection.create_index("embedding", ivf_params)
collection.load()
start = time.time()
results = collection.search([np.random.rand(128).tolist()], "
   embedding", param={"metric_type": "COSINE", "params": {"nprobe"
   : 10}}, limit=10)
print("IVF tiempo:", time.time() - start)
```

Se observará que HNSW suele ser más rápido pero consume más memoria.

Capítulo 11

Glosario de Términos

Embedding Representación vectorial de un objeto (texto, imagen, etc.) en un espacio continuo.

Vector Lista ordenada de números que representa un embedding.

Dimensionalidad Número de componentes del vector (ej. 384, 768, 1536).

Similitud de coseno Medida de similitud basada en el ángulo entre vectores.

Distancia euclidiana Distancia geométrica entre vectores.

ANN Approximate Nearest Neighbors, algoritmos para búsqueda aproximada de vecinos cercanos.

HNSW Hierarchical Navigable Small World, algoritmo de indexación basado en grafos.

IVF Inverted File Index, algoritmo que divide el espacio en clusters.

RAG Retrieval Augmented Generation, técnica que combina recuperación de información y generación de texto.

LangChain Framework para construir aplicaciones con LLMs.

LlamaIndex Framework para indexación y recuperación de datos para LLMs.

pgvector Extensión de PostgreSQL para vectores.

Pinecone Base de datos vectorial gestionada en la nube.

Milvus Plataforma de código abierto para búsqueda vectorial.

Weaviate Base de datos vectorial de código abierto con módulos de ML.

Qdrant Base de datos vectorial de alto rendimiento.

Referencias

- Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. (Faiss paper).
- Malkov, Y. A., & Yashunin, D. A. (2018). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. (HNSW paper).
- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. (RAG paper).
- Documentación de Sentence Transformers: `https://www.sbert.net/`
- Documentación de pgvector: `https://github.com/pgvector/pgvector`
- Documentación de LangChain: `https://python.langchain.com/`
- Documentación de Milvus: `https://milvus.io/docs`
- Documentación de Pinecone: `https://docs.pinecone.io/`